



中國人民大學
RENMIN UNIVERSITY OF CHINA

程序设计荣誉课程

8 标准模板库STL

授课教师：孙亚辉

标准模板库

- 标准模板库（Standard Template Library, STL）提供了一组表示容器、迭代器、函数对象和算法的模板
 - 容器（Container）：作为某类对象的集合
 - 迭代器（Iterator）：允许在对象集合上进行遍历
 - 算法（Algorithm）：处理集合内的元素的方法
- 基础组件间的关系
 - 容器提供迭代器，算法通过迭代器操作容器里的元素
 - 迭代器使得容器和算法相互独立



本章内容

1. 标准模板库的容器
2. 标准模板库的迭代器
3. 标准模板库的算法

1. 容器

- 容器是可以用于存放各种类型的数据（基本类型的变量，对象等）的数据结构。
 - 可理解为：容器是一个与数组类似的单元，可以存储若干个值
- `string`可视作存储字符数据的“容器”。
 - `string`是针对`char`类型的`basic_string`类模板的显式实例化

```
template <typename CharT, typename Traits = char_traits<CharT>,
typename Alloc = allocator<CharT> >
class basic_string {
    ....
};
typedef basic_string<char>      string; // char是8位字符类型，最多只
能包含256种字符
typedef basic_string<wchar_t> wstring; // wchar_t数据类型一般为16
位或32位，wchar_t所能表示的字符数远超char型
```

- string可视作存储字符数据的“容器”
- string是针对char类型的basic_string类模板的显式实例化

Microsoft's C++ Standard Library:

<https://github.com/microsoft/STL>

STL-main\stl\inc\xstring:

```
template <class _Elem, class _Traits = char_traits<_Elem>, class _Alloc = allocator<_Elem>>
class basic_string { // null-terminated transparent array of elements
private:
    friend _Tidy_deallocate_guard<basic_string>;
    friend basic_stringbuf<_Elem, _Traits, _Alloc>;

    using _Alt_v      = _Rebind_allocator<_Alloc, _Elem>;
```

```
    using string = basic_string<char>;
#ifdef __cpp_lib_char8_t
    using u8string = basic_string<char8_t>;
#endif // __cpp_lib_char8_t
    using u16string = basic_string<char16_t>;
    using u32string = basic_string<char32_t>;
    using wstring   = basic_string<wchar_t>;
    ...
```

容器的分类

- 顺序容器
 - vector: 一般尾部插入/删除, 直接访问
 - deque: 一般首/尾部插入/删除, 直接访问
 - list: 双向链表, 任意位置插入/删除
- 关联容器
 - set: 快速查找, 无重复元素 (一般用红黑树实现)
 - multiset: 快速查找, 可有重复元素
 - map: 一一映射, 无重复元素, 基于关键字查找 (Set和Map的区别在于Set只含有Key, 而Map有一个Key和Key所对应的Value两个元素)
 - multimap: 一一映射, 可有重复元素, 基于关键字查找
- 容器适配器 (虽然适配器中也可以存放元素, 但适配器只是对其他容器的接口进行了包装, 比如STL中stack和queue默认使用deque)
 - stack: 后进先出
 - queue: 先进先出
 - priority_queue: 优先级最高的元素先出

STL容器的共有成员函数

- `empty` : 判断容器中是否有元素
- `size`: 容器中元素个数
- `capacity`: 容器中最多能装多少元素
- `swap`: 交换两个容器的内容
- 相当于按词典顺序比较两个容器大小的运算符:
`<` , `<=` , `>` , `>=` , `==` , `!=`

比较两个容器的例子

```
#include <vector>
#include <iostream>
using namespace std;
int main()
{
    std::vector<int> v1;
    std::vector<int> v2;
    v1.push_back (5);
    v1.push_back (1);
    v2.push_back (1);
    v2.push_back (2);
    v2.push_back (3);
    cout << (v1 < v2);
    return 0;
}
```

- 若两容器长度相同、所有元素相等，则两个容器就相等，否则为不等
- 若两容器长度不同，但较短容器中所有元素都等于较长容器中对应的元素，则较短容器小于另一个容器
- 若两个容器均不是对方的子序列，则取决于所比较的第一个不等的元素

输出结果: 0

1.1 顺序容器

- 特性：顺序容器的元素排列和元素值大小无关，而是由元素添加到容器中的次序决定的。
- 不同种类的顺序容器
 - **vector**：实际上是个动态数组。随机读取任何元素都能在**常数时间**完成。在**尾端增删元素**具有较佳的性能。
 - **deque**（双端队列）：结构比较复杂，由多个动态数组组成，随机读取任何元素、在两端增删元素都能在**常数时间**完成。
 - **list**：双向链表，在**任何位置增删元素**都能在**常数时间**完成。只支持双向顺序访问，不支持随机读取。

```
// vector
```

```
reference operator[](size_type __n) {  
    __glibcxx_require_subscript(__n);  
    return *(this->_M_impl._M_start + __n);  
}
```

```
// deque
```

```
reference operator[](size_type __n) {  
    __glibcxx_require_subscript(__n);  
    return this->_M_impl._M_start[difference_type(__n)];  
}
```

```
// list
```

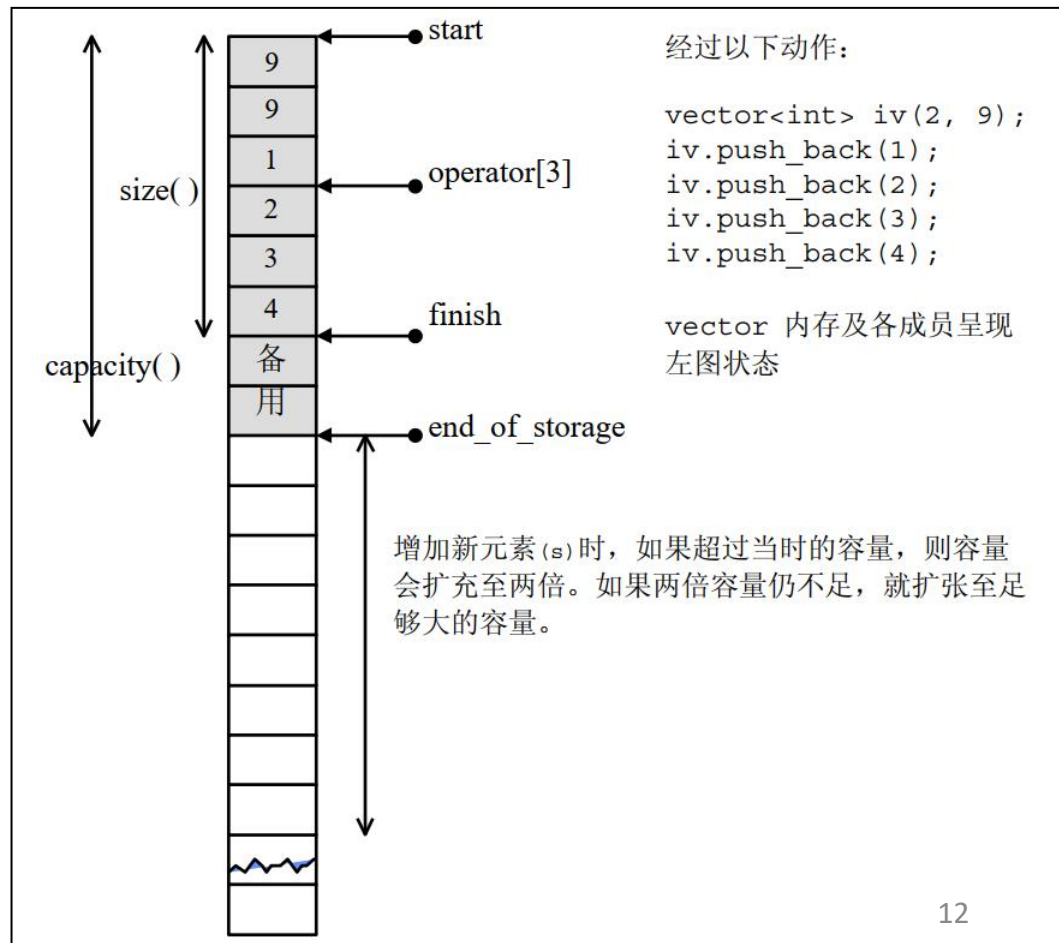
```
// 未重载operator[] （list不能使用[]）
```

顺序容器的共有成员函数

- `front()`: 返回容器中第一个元素的引用
- `back()`: 返回容器中最后一个元素的引用
- `push_back()`: 在容器末尾增加新元素
- `pop_back()`: 删除容器末尾的元素——无返回值
- `begin()`: 返回指向容器中第一个元素的迭代器
- `end()`: 返回指向容器中最后一个元素后面位置的迭代器
- `rbegin()`: 返回指向容器中最后一个元素的迭代器
- `rend()`: 返回指向容器中第一个元素前面位置的迭代器
- `erase()`: 从容器中删除一个或几个元素
- `clear()`: 从容器中删除所有元素（但容器的大小或容量不变）

vector容器

- 向量容器的构造：`vector<type>` 容器变量名;
- 往向量容器放置元素
 - `push_back`, `insert`
 - `resize()`改变size大小
- 删除向量容器中的元素
 - `pop_back`, `erase`, `clear`
- 访问向量容器中的元素
 - `front`, `back`
 - 下标[]运算符、`at()`
 - 使用迭代器



vector内部实现有三个指针：

- pointer _M_start;
- pointer _M_finish;
- pointer _M_end_of_storage;

分别代表其使用内存的开始、结束，以及分配的内存结尾，其中

_M_finish - _M_start 对应 size();

_M_end_of_storage - _M_start 对应 capacity() ;

在64位机器上，vector容器占用24位（3个指针；注意sizeof与size的区别；后续讲的deque容器占用更多内存空间）。

```
1  #include <iostream>
2  #include <vector>
3  #include <deque>
4  #include <list>
5  using namespace std;
6
7  int main()
8  {
9      vector<int> v1(100);
10     vector<double> v2;
11     deque<int> d1;
12     list<int> l1;
13
14     cout << sizeof(v1) << " " << v1.size() << endl;
15     cout << sizeof(v2) << " " << v2.size() << endl;
16     cout << sizeof(d1) << endl;
17     cout << sizeof(l1) << endl;
18 }
```

```
yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
24 100
24 0
80
24
```

```
#include<iostream>
#include<vector>
using namespace std;

struct Student
{
    string number;
    string name;
    string sex;
    float score1,score2,score3,aver;
};

vector<Student*> StuVec;
```

```
void addStu() {
    Student* stu;
    char choi;

    while(true) {
        cout << "\t您确定要添加一位学生的信息吗?(y/n)";
        cin >> choi;
        if(choi=='n' || choi=='N')
            break;
        stu = new Student;
        cout << "\t学号" << "\t姓名" << "\t性别" << "\t语文"
            << "\t数学" << "\t英语" << endl << "\t";
        cin >> stu->number;
        cin >> stu->name;
        cin >> stu->sex;
        cin >> stu->score1;
        cin >> stu->score2;
        cin >> stu->score3;
        stu->aver = (stu->score1 + stu->score2 + stu->score3) / 3;
        StuVec.push_back(stu);
    }
}
```

```
void Display()
```

```
{
```

```
    vector<Student*>::iterator it;
```

```
    .....
```

```
    for(it=StuVec.begin(); it!=StuVec.end(); ++it )
```

```
        cout <<"\t"<<(*it)->number<<"\t"<<(*it)->name
```

```
            <<"\t"<<(*it)->sex<<"\t"<<(*it)->score1
```

```
            <<"\t"<<(*it)->score2<<"\t"<<(*it)->score3
```

```
            <<"\t"<<(*it)->aver<<endl;
```

```
}
```

通过迭代器，顺序遍历vector中所有元素——迭代器得到的是元素的地址，需要解应用（如*it）才能使用

```
void Display()
```

```
{
```

```
    for(int i=0; i < StuVec.size(); i++) {
```

```
        Student* it = StuVec[i];
```

```
        cout <<"\t"<<it->number<<"\t"<<it->name
```

```
            <<"\t"<<it->sex<<"\t"<<it->score1
```

```
            <<"\t"<<it->score2<<"\t"<<it->score3
```

```
            <<"\t"<<it->aver<<endl;
```

```
    }
```

```
}
```

通过传统的数组访问方式来访问
vector中的元素

```
void Display()
```

```
{
```

```
    for(Student* stu : StuVec)
```

```
        cout <<"\t"<< stu->number . . . . .
```

```
}
```

使用基于范围的for循环，每次遍历得到的元素就已经是Student*类型

通过传统的数组访问方式来访问vector中的元素时，尽量少调用size()函数

```
1  #include <iostream>
2  #include <vector>
3  #include <chrono>
4  using namespace std;
5  int main()
6  {
7
8      vector<int> x(1e7, 1);
9
10     if (1)
11     {
12         auto begin = std::chrono::high_resolution_clock::now();
13
14         for (int i = 0; i < x.size(); i++)
15         {
16
17
18         }
19
20         auto end = std::chrono::high_resolution_clock::now();
21         float runningtime = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count() / 1e6;
22         std::cout << runningtime << "ms" << std::endl;
23     }
24     if (1)
25     {
26         auto begin = std::chrono::high_resolution_clock::now();
27
28         for (int i = x.size() - 1; i >= 0; i--)
29         {
30
31
32         }
33
34         auto end = std::chrono::high_resolution_clock::now();
35         float runningtime = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count() / 1e6;
36         std::cout << runningtime << "ms" << std::endl;
37     }
38 }
```

```
PS C:\Users\Yahui\Drive\Documents> g++ test.cpp
23.395ms
16.591ms
```

```
yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
48.6871ms
16.5901ms
```

频繁地读取vector中的某个元素时，使用指针访问该元素较[]快

```
1  #include <vector>
2  #include <iostream>
3  #include <chrono>
4  using namespace std;
5
6  void no_pointer(int times, vector<int> &x)
7  {
8      auto begin = std::chrono::high_resolution_clock::now();
9
10     for (int i = 0; i < times; i++)
11     {
12         x[10];
13     }
14
15     auto end = std::chrono::high_resolution_clock::now();
16     double runningtime = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count() / 1e6;
17     cout << "no_pointer runningtime: " << runningtime << "ms" << endl;
18 }
19
20 void with_pointer(int times, vector<int> &x)
21 {
22     auto begin = std::chrono::high_resolution_clock::now();
23
24     auto it = &x[10];
25     for (int i = 0; i < times; i++)
26     {
27         *it;
28     }
29
30     auto end = std::chrono::high_resolution_clock::now();
31     double runningtime = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count() / 1e6;
32     cout << "with_pointer runningtime: " << runningtime << "ms" << endl;
33 }
34
35 int main()
36 {
37     int vector_size = 1e2, times = 1e8;
38     vector<int> x(vector_size);
39     no_pointer(times, x);
40     with_pointer(times, x);
41     return 0;
42 }
```

```
PS C:\Users\Yahui\Drive\VM> cd "c:\u
no_pointer runningtime: 229.36ms
with_pointer runningtime: 220.589ms
```

```
yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
no_pointer runningtime: 551.214ms
with_pointer runningtime: 228.578ms
```

clear不改变vector等容器的大小（capacity）；**建议用swap将容器置空。**
swap清空方法：假如a的类型是X，则 **X().swap(a)**；清空a。

```
1  #include <iostream>
2  #include <vector>
3  #include <random>
4  using namespace std;
5
6  #include <Current_Memory_Consumption_of_This_Process.h>
7
8  int main()
9  {
10     vector<int> a(1e9);
11     cout << Current_Memory_Consumption_of_This_Process() << " MB" << endl;
12     a.clear();
13     cout << a.capacity() << endl;
14     cout << Current_Memory_Consumption_of_This_Process() << " MB" << endl;
15     vector<int>().swap(a);
16     cout << Current_Memory_Consumption_of_This_Process() << " MB" << endl;
17     return 0;
18 }
```

```
root@user-PowerEdge-R750:~# sh run.sh
3817.66 MB
1000000000
3817.66 MB
3.28125 MB
```

swap可以清除容器的多余空间。

swap清除容器的多余空间的方法：假如容器a的类型是X，则X(a).swap(a);清除a的多余空间。

```
1  #include <iostream>
2  #include <string>
3  #include <vector>
4  using namespace std;
5
6  int main()
7  {
8      vector<int> v;
9
10     for (int i = 0; i < 9; i++)
11     {
12         v.push_back(i);
13     }
14
15     cout << "v.size(): " << v.size() << endl;
16     cout << "v.capacity(): " << v.capacity() << endl;
17
18     vector<int>(v).swap(v);
19
20     cout << "v.size(): " << v.size() << endl;
21     cout << "v.capacity(): " << v.capacity() << endl;
22
23 }
```

```
yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
v.size(): 9
v.capacity(): 16
v.size(): 9
v.capacity(): 9
```

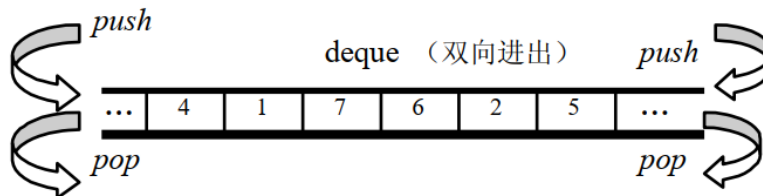
resize不缩小**vector**等容器的大小（capacity），但可增大容器的大小。

```
1  #include <iostream>
2  #include <string>
3  #include <vector>
4  using namespace std;
5
6  int main()
7  {
8      vector<int> v;
9
10     for (int i = 0; i < 9; i++)
11     {
12         v.push_back(i);
13     }
14
15     cout << "v.size(): " << v.size() << endl;
16     cout << "v.capacity(): " << v.capacity() << endl;
17
18     v.resize(5);
19
20     cout << "v.size(): " << v.size() << endl;
21     cout << "v.capacity(): " << v.capacity() << endl;
22
23     v.resize(17);
24
25     cout << "v.size(): " << v.size() << endl;
26     cout << "v.capacity(): " << v.capacity() << endl;
27 }
```

```
yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
v.size(): 9
v.capacity(): 16
v.size(): 5
v.capacity(): 16
v.size(): 17
v.capacity(): 17
```

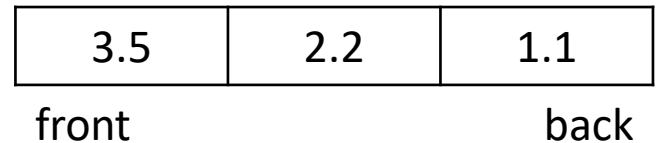
deque容器

- 双端队列是一种放松了访问权限的队列。元素可从队列的两端入队和出队。
- 使用双端队列容器
 - 向双端队列容器放置元素
 - `push_back`, `insert`, 运算符 “`=`”, 下标`[]`运算符
 - `push_front`
 - 删除双端队列容器中的元素
 - `pop_front`, `pop_back`, `erase`, `clear`
 - 访问双端队列容器中的元素
 - `front`, `back`, 下标`[]`运算符, `at`
 - 使用迭代器



```
#include<iostream>
#include<deque>
#include<algorithm>
using namespace std;
int main()
{
    deque<double> de, value;
    deque<double>::iterator it;
    de.push_front(2.2);
    de.push_front(3.5);
    de.push_back(1.1);
    for(int i=0; i<de.size(); i++)
        cout<< de[i] <<" ";
    cout<<endl;
    de.pop_front();

    value=de;
    for(it=value.begin(); it!=value.end(); ++it )
        cout<< *it <<" ";
}
```



deque_vs_vector

```
36         if (1)
37         {
38             auto begin = std::chrono::high_resolution_clock::now();
39             for (int i = 0; i < max_time; i++)
40             {
41                 de[i / 2];
42             }
43             auto end = std::chrono::high_resolution_clock::now();
44             double runningtime = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count(); // Nanosecond
45             std::cout << "de runningtime:" << runningtime / 1e6 << "ms" << std::endl;
46         }
47         if (1)
48         {
49             auto begin = std::chrono::high_resolution_clock::now();
50             for (int i = 0; i < max_time; i++)
51             {
52                 ve[i / 2];
53             }
54             auto end = std::chrono::high_resolution_clock::now();
55             double runningtime = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count(); // Nanosecond
56             std::cout << "ve runningtime:" << runningtime / 1e6 << "ms" << std::endl;
57         }
58     }
59 }

7 int main()
8 {
9     deque<double> de;
10    vector<double> ve;
11    int max_time = 5e4;
12    if (1)
13    {
14        auto begin = std::chrono::high_resolution_clock::now();
15        for (int i = 0; i < max_time; i++)
16        {
17            de.push_front(i);
18            de.push_back(i);
19        }
20        auto end = std::chrono::high_resolution_clock::now();
21        double runningtime = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count(); // Nanosecond
22        std::cout << "de runningtime:" << runningtime / 1e6 << "ms" << std::endl;
23    }
24    if (1)
25    {
26        auto begin = std::chrono::high_resolution_clock::now();
27        for (int i = 0; i < max_time; i++)
28        {
29            ve.insert(ve.begin(), i);
30            ve.push_back(i);
31        }
32        auto end = std::chrono::high_resolution_clock::now();
33        double runningtime = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count(); // Nanosecond
34        std::cout << "ve runningtime:" << runningtime / 1e6 << "ms" << std::endl;
35    }
36 }
```

```
PS C:\Users\Yahui\Drive\VM>
de runningtime:2.961ms
ve runningtime:1683.63ms
de runningtime:1.951ms
ve runningtime:0ms

yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
de runningtime:3.70959ms
ve runningtime:508.649ms
de runningtime:3.87658ms
ve runningtime:0.249519ms
```

list容器

- 在任何位置插入删除都是常数时间，不支持随机读取。部分成员函数：
 - `push_front`: 在前面插入
 - `pop_front`: 删除前面的元素
 - `sort`: 排序
 - `remove`: 删除和指定值相等的所有元素
 - `unique`: 删除所有和前一个元素相同的元素
 - `merge`: 合并两个有序链表，并清空被合并的那个
 - `reverse`: 颠倒链表
 - `splice`: 在指定位置插入另一链表中的一个或多个元素，并在另一链表中删除被插入的元素

例：从键盘输入10个整数，用这些整数值作为结点数据，**生成一个链表**，**按顺序输出**链表中结点的数值。然后从键盘输入一个待查找整数，在链表中**查找该整数**，若找到则**删除该整数所在的结点**（如果出现多次，全部删除），然后输出删除结点以后的链表。在程序结束之前清空链表。

```
1  #include <iostream>
2  #include <list>
3  using namespace std;
4
5  int main() {
6      list<int> Link;    // 构造一个列表用于存放整数链表
7      int i, key, item;
8      for(i=0; i < 10; i++) {    // 输入10个整数依次向表头插入
9          cin>>item;
10         Link.push_front(item);
11     }
12     cout<<"List: ";    // 输出链表
13     list<int>::iterator p=Link.begin();
14     while(p!=Link.end()) {    //输出各节点数据，直到链表尾
15         cout <<*p << " ";
16         p++;    //使P指向下一个节点
17     }
18     cout << "\n请输入一个需要删除的整数: ";
19     cin >> key;
20     Link.remove(key);    //void remove(const T& x)
21     cout << "List: ";    // 输出链表
22     p=Link.begin();    // 使p重新指向表头
23     while(p!=Link.end()) {
24         cout <<*p << " ";
25         p++;
26     }
27     cout << endl;
28 }
```

`list::unique()` 的用处：删除所有和前一个元素相同的元素（Removes all consecutive duplicate elements from the container. Only the first element in each group of equal elements is left.）

```
1  #include <iostream>
2  #include <list>
3
4  template <class T>
5  auto print(T const &container)
6  {
7      for (auto const &val : container)
8          std::cout << ' ' << val;
9      std::cout << '\n';
10 };
11
12 int main()
13 {
14     std::list<int> c = {1, 2, 2, 3, 3, 2, 1, 1, 2};
15     print(c);
16     c.unique();
17     print(c);
18 }
```

```
yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
1 2 2 3 3 2 1 1 2
1 2 3 2 1 2
```

自定义list unique() 的比较函数:

```
1  #include <iostream>
2  #include <list>
3  using namespace std;
4
5  // Function for binary_predicate
6  bool compare(double a, double b)
7  {
8      return ((int)a == (int)b);
9  }
10
11 // Driver code
12 int main()
13 {
14     list<double> list = {2.55, 3.15, 4.16, 4.16,
15 | | | | | | | | 4.77, 12.65, 12.65, 13.59};
16     cout << "List is: ";
17     // sort the list
18     list.sort();
19     // unique operation on list with no parameters
20     list.unique();
21     // starts from the first element
22     // of the list to the last
23     for (auto it = list.begin(); it != list.end(); ++it)
24         cout << *it << " ";
25     // unique operation on list with parameter
26     list.unique(compare);
27     cout << "\nList is: ";
28     // starts from the first element
29     // of the list to the last
30     for (auto it = list.begin(); it != list.end(); ++it)
31         cout << *it << " ";
32     cout << endl;
33     return 0;
34 }
```

```
yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
List is: 2.55 3.15 4.16 4.77 12.65 13.59
List is: 2.55 3.15 4.16 12.65 13.59
```

list merge() 的用处：合并两个有序链表，并清空被合并的那个（其不会校验待合并链表的排序正确性）

```
1  #include <iostream>
2  #include <list>
3  using namespace std;
4
5  using namespace std;
6
7  int main()
8  {
9      list<int> list1 = {10, 21, 40};
10     list<int> list2 = {30, 50, 60};
11     // merge operation
12     list2.merge(list1);
13
14     cout << "List: ";
15     for (auto it = list2.begin(); it != list2.end(); ++it)
16         cout << *it << " ";
17     cout << endl;
18     return 0;
19 }
```

```
yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
List:  10 21 30 40 50 60
```

```

1  #include <iostream>
2  #include <list>
3  using namespace std;
4
5  using namespace std;
6
7  int main()
8  {
9      list<int> list1 = {10, 51, 40};
10     list<int> list2 = {30, 70, 60};
11     // merge operation
12     list2.merge(list1);
13
14     cout << "List: ";
15     for (auto it = list2.begin(); it != list2.end(); ++it)
16     |   cout << *it << " ";
17     cout << endl;
18     return 0;
19 }

```

```

yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
List:  10 30 51 40 70 60

```

```

std::ostream& operator<<(std::ostream& ostr, const std::list<int>& list) {
    for (auto &i : list) {
        ostr << " " << i;
    }
    return ostr;
}

int main() {
    std::list<int> list = { 8,7,5,9,0,1,3,2,6,4 };

    std::cout << "before:      " << list << "\n";
    list.sort();
    std::cout << "ascending:  " << list << "\n";
    list.reverse();
    std::cout << "descending: " << list << "\n";
}

```

输出结果:

```

before:      8 7 5 9 0 1 3 2 6 4
ascending:   0 1 2 3 4 5 6 7 8 9
descending:  9 8 7 6 5 4 3 2 1 0

```

```

std::ostream& operator<<(std::ostream& ostr, const std::list<int>& list) {
    for (auto &i : list) {
        ostr << " " << i;
    }
    return ostr;
}

```

```

int main () {
    std::list<int> list1 = { 1, 2, 3, 4, 5 };
    std::list<int> list2 = { 10, 20, 30, 40, 50 };

```

```

    auto it = list1.begin();
    std::advance(it, 2);

```

```

    list1.splice(it, list2);

```

```

    std::cout << "list1: " << list1 << "\n";
    std::cout << "list2: " << list2 << "\n";
    // 此时it指向list1的3

```

```

    list2.splice(list2.begin(), list1, it, list1.end());

```

```

    std::cout << "list1: " << list1 << "\n";
    std::cout << "list2: " << list2 << "\n";

```

```

}

```

输出结果:

list1: 1 2 10 20 30 40 50 3 4 5

list2:

list1: 1 2 10 20 30 40 50

list2: 3 4 5

std::bitset

std::bitset类型表示一个固定长度的位序列，每个位都只能是0或1。std::bitset提供了一些常规操作，比如

- count()返回std::bitset中值为1的位的数量
- size()返回std::bitset的长度

std::bitset重载了许多二进制运算符，如&、|、^（异或）、~、<<、>>等，使其支持类似于整数类型的位运算操作。

```

1  #include <iostream>
2  #include <bitset>
3  using namespace std;
4
5  int main()
6  {
7      bitset<4> a1{ "0011" };
8      bitset<4> a2 = 4; // 0100
9      bitset<4> a3;
10     a3[0] = 0, a3[1] = 1, a3[2] = 0, a3[3] = 1; // 1010    a3[0]是最右端的一位
11
12     cout << a1 << " " << a2 << " " << a3 << " " << a1.count() << " " << a1.size() << endl;
13     cout << (a1 & a2) << " " << (a1 & a3) << endl; //与运算
14     cout << (a1 | a2) << " " << (a1 | a3) << endl; //或运算
15     cout << (a1 ^ a2) << " " << (a1 ^ a3) << endl; //异或运算
16     cout << ~a1 << " " << ~a2 << endl; //取反运算
17     cout << (a1 >> 2) << " " << (a2 << 1) << endl; //移位运算
18 }

```

Microsoft Visual Studio Debug Console

```

0011 0100 1010 2 4
0000 0010
0111 1011
0111 1001
1100 1011
0000 1000

```

1.2 关联容器

- 关联容器（associative container）将键（key）与值（value）关联在一起。
- 特性：关联式容器内的元素是排序的，插入任何元素，都按相应的排序准则来确定其位置。关联式容器的特点是在查找时具有非常好的性能。
- 不同种类的关联容器
 - set/multiset：集合。set中不允许相同元素，multiset中允许存在相同的元素
 - map/multimap：存放成对key/value，根据key对元素进行排序，可快速根据key检索元素。map同multimap的不同在于是否允许多个元素有相同的key

关联容器的共有成员函数

- `find(k)`: 若容器中存在键为k的元素，则返回指向该元素的迭代器。否
则返回`end()`值
- `lower_bound(k)`: 返回一个迭代器，指向键大于或等于k的第一个元素
 - 对于set来说，值（value）就是键（key）
- `upper_bound(k)`: 返回一个迭代器，指向键大于k的第一个元素
- `m.count(k)`: 返回容器m中k的出现次数
- `insert()`: 插入元素
- `erase()`: 删除指定位置、区域、或键值的元素

set/multiset容器

- 集合和多重集合提供了控制数值集合的操作，其中数值是关键字（key），即不必另有一组值与每个关键字相关联。
- 多重集合multiset允许重复的关键字（key），而集合set不允许
- 元素的顺序由比较器函数对象（comparator function object）确定。如对整型set/multiset，只要用比较器函数对象less<int>为排序关键字，元素即可按升序排列

- 默认升序排列

```
template<typename _Tp>
struct less : public binary_function<_Tp, _Tp, bool> {
    bool operator()(const _Tp& __x, const _Tp& __y) const
    {
        return __x < __y;
    }
};
```

std::set

Defined in header `<set>`

```
template<
    class Key,
    class Compare = std::less<Key>,
    class Allocator = std::allocator<Key>
> class set;                                     (1)

namespace pmr {
    template <class Key, class Compare = std::less<Key>>
        using set = std::set<Key, Compare, std::pmr::polymorphic_allocator<Key>>;    (2) (since C++17)
}
```

`std::set` is an associative container that contains a sorted set of unique objects of type `Key`. Sorting is done using the key comparison function `Compare`. Search, removal, and insertion operations have logarithmic complexity. Sets are usually implemented as red-black trees.

```

1 #include <iostream>
2 #include <set>
3 using namespace std;
4
5 bool fncomp(int lhs, int rhs)
6 {
7     return lhs > rhs; // < is from small to large, > is from large to small
8 }
9
10 struct classcomp
11 {
12     bool operator()(const int &lhs, const int &rhs) const
13     {
14         return lhs > rhs; // < is from small to large, > is from large to small
15     }
16 };
17
18 void print_set(std::set<int> &s)
19 {
20     for (auto it = s.begin(); it != s.end(); it++)
21     {
22         cout << *it << " ";
23     }
24     cout << endl;
25 }
26
27 template<typename T>
28 void print_set(std::set<int, T> &s)
29 {
30     for (auto it = s.begin(); it != s.end(); it++)
31     {
32         cout << *it << " ";
33     }
34     cout << endl;
35 }

```

函数对象（注意参数与函数类型为const）

```

yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
1 2 4
10 20 31 40 50
10 20 31 40 50
10 20 31 40 50
4 2 1
4 2 1

```

```

37 int main()
38 {
39     std::set<int> first = {2, 4, 1}; // empty set of ints
40     print_set(first);
41
42     int myints[] = {10, 31, 20, 20, 50, 40};
43     std::set<int> second(myints, myints + 6); // range; insert 6 elements
44     print_set(second);
45
46     std::set<int> third(second); // a copy of second
47     print_set(third);
48
49     std::set<int> fourth(second.begin(), second.end()); // iterator ctor.
50     print_set(fourth);
51
52     std::set<int, classcomp> fifth = {2, 4, 1}; // class as Compare
53     print_set(fifth);
54
55     bool (*fn_pt)(int, int) = fncomp; // 函数指针
56     std::set<int, bool (*)(int, int)> sixth(fn_pt); // function pointer as Compare
57     sixth.insert(2); sixth.insert(4); sixth.insert(1);
58     print_set(sixth);
59
60     return 0;
61 }

```

用函数对象、函数指针初始化一个内部元素为int的set类型的变量

自定义排序函数1

```
struct intComp {
    bool operator() (const int& lhs, const int& rhs) const {
        return lhs > rhs;
    }
};

struct strComp {
    bool operator() (const string& str1, const string& str2) {
        return str1.length() < str2.length();
    }
};

int main() {
    set<int, intComp> s1 = {10, 20, 30, 40, 50};
    for (int i : s1) cout << i << " ";
    cout << endl;
    set<string, strComp> s2 = {"hello", "world", "boy", "girl"};
    for (const string& str : s2) cout << str << " ";
    cout << endl;
    return 0;
}
```

输出结果:

50 40 30 20 10

boy girl hello

(没有world, 因为和hello的键相同)

```

struct FatKey    { int x; int data[1000]; };
struct LightKey { int x; };
bool operator<(const FatKey& fk, const LightKey& lk) { return fk.x < lk.x; }
bool operator<(const LightKey& lk, const FatKey& fk) { return lk.x < fk.x; }
bool operator<(const FatKey& fk1, const FatKey& fk2) { return fk1.x < fk2.x; }

int main() {
    // simple comparison demo
    std::set<int> example = {1, 2, 3, 4};
    auto search = example.find(2);
    if (search != example.end()) {
        std::cout << "Found " << (*search) << '\n';
    } else {
        std::cout << "Not found\n";
    }
    // transparent comparison demo
    std::set<FatKey, std::less<>> example2 = { {1, {} } , {2, {} } , {3, {} } , {4, {} } };
    LightKey lk = {2};
    auto search2 = example2.find(lk); // 此处需要重载的<操作符
    if (search2 != example2.end()) {
        std::cout << "Found " << search2->x << '\n';
    } else {
        std::cout << "Not found\n";
    }
}

```

自定义排序函数2

输出结果:
Found 2
Found 2

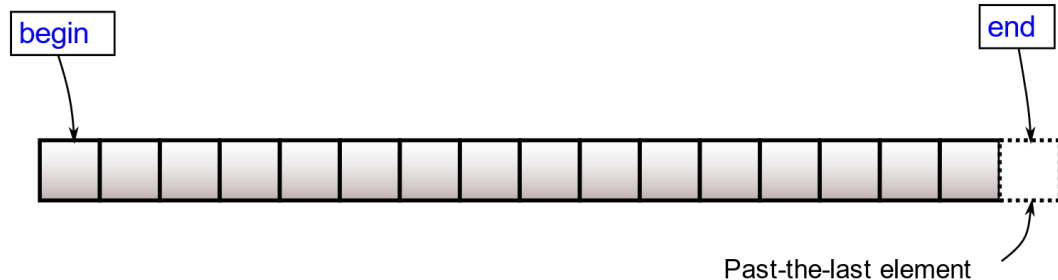
```

int main() {

    std::set<int> c = {1, 2, 3, 4, 5, 6, 7, 8, 9};

    // erase all odd numbers from c
    for(auto it = c.begin(); it != c.end(); ) {
        if(*it % 2 != 0)
            it = c.erase(it);
        else
            ++it;
    }
    for(int n : c) {
        std::cout << n << ' ';
    }
}

```



输出结果:
2 4 6 8

pair模板

pair模板类用来绑定两个对象为一个新的对象。pair模板类支持如下操作：

- `pair<T1, T2> p1`: 创建一个空的pair对象，它的两个元素分别是T1和T2类型，采用值初始化
- `pair<T1, T2> p1(v1, v2)`: 创建一个pair对象，它的两个元素分别是T1和T2类型，其中first成员初始化为v1，second成员初始化为v2
- `make_pair(v1, v2)`: 以v1和v2值创建一个新的pair对象，其元素类型分别是v1和v2的类型
- `p1 < p2` 字典次序：如果 `p1.first < p2.first` 或者 `(p2.first == p1.first) && p1.second < p2.second`，则返回true
- `p1 == p2`: 如果两个pair对象的first和second成员依次相等，则这两个对象相等。
- `p.first`: 返回p中名为first的（公有）数据成员
- `p.second`: 返回p中名为second的（公有）数据成员

```

1  #include <vector>
2  #include <iostream>
3  #include <set>
4  using namespace std;
5
6  int main()
7  {
8      typedef set<double, less<double>> double_set; // less is a compare function (default from small to large)
9
10     const int SIZE = 5;
11     double a[SIZE] = {2.1, 4.2, 9.5, 2.1, 3.7};
12
13     double_set doubleSet(a, a + SIZE); // a is an address; doubleSet is a set containing all the above identical elements
14
15     for (auto it = doubleSet.begin(); it != doubleSet.end(); it++)
16     {
17         cout << *it << endl;
18     }
19
20     pair<double_set::const_iterator, bool> p = doubleSet.insert(9.5); // 9.5 is already inside, so is not inserted
21
22     if (p.second)
23         cout << "1) " << *(p.first) << " inserted" << endl;
24     else
25         cout << "2) " << *(p.first) << " not inserted" << endl;
26     return 0;
27 }

```

```

yahooi@Yahooi:/media/sf_VM$ g++ test.cpp
yahooi@Yahooi:/media/sf_VM$ ./a.out
2.1
3.7
4.2
9.5
2) 9.5 not inserted

```

`insert`函数返回值是一个`pair`对象，其`first`是被插入元素的迭代器，`second`代表是否成功插入了

- `second`为`true`，则`first`指向新插入的元素
- `second`为`false`，则`first`指向阻止插入操作的元素

map/multimap容器

- 提供了操作与关键字相关联的映射值（mapped value）的方法，能自动按关键字进行排序
- 映射map和多重映射multimap的主要差别在于：
 - 多重映射multimap允许存放与映射值相关联的重复关键字，而映射map只允许存放与映射值一一对应的单一关键字。
- 映射map和多重映射multimap用于快速存储和读取关键字与相关值（关键字/数值对，key/value pair）

```

1  #include <iostream>
2  #include <iterator>
3  #include <map>
4
5  using namespace std;
6
7  int main()
8  {
9      // empty map container
10     map<int, int> gquiz1;
11     // insert elements in random order
12     gquiz1.insert(pair<int, int>(1, 40));
13     gquiz1.insert(pair<int, int>(2, 30));
14     gquiz1.insert(pair<int, int>(3, 60));
15     gquiz1.insert(pair<int, int>(4, 20));
16     gquiz1.insert(pair<int, int>(5, 50));
17     gquiz1.insert(pair<int, int>(6, 50));
18     gquiz1.insert(pair<int, int>(7, 10));
19     // printing map gquiz1
20     map<int, int>::iterator itr;
21     cout << "\nThe map gquiz1 is : \n"
22     | | | << "\tKEY\tELEMENT\n";
23     for (itr = gquiz1.begin(); itr != gquiz1.end(); ++itr)
24     {
25         cout << '\t' << itr->first
26         | | | << '\t' << itr->second << '\n';
27     }
28     cout << endl;

```

```

30     // assigning the elements from gquiz1 to gquiz2
31     map<int, int> gquiz2(gquiz1.begin(), gquiz1.end());
32     // print all elements of the map gquiz2
33     cout << "\nThe map gquiz2 after"
34     | | | << " assign from gquiz1 is : \n";
35     cout << "\tKEY\tELEMENT\n";
36     for (itr = gquiz2.begin(); itr != gquiz2.end(); ++itr)
37     {
38         cout << '\t' << itr->first
39         | | | << '\t' << itr->second << '\n';
40     }
41     cout << endl;
42     // remove all elements up to
43     // element with key=3 in gquiz2
44     cout << "\ngquiz2 after removal of elements less than key=3 : \n";
45     cout << "\tKEY\tELEMENT\n";
46     gquiz2.erase(gquiz2.begin(), gquiz2.find(3));
47     for (itr = gquiz2.begin(); itr != gquiz2.end(); ++itr)
48     {
49         cout << '\t' << itr->first
50         | | | << '\t' << itr->second << '\n';
51     }
52     // remove all elements with key = 4
53     int num = gquiz2.erase(4); // 返回去掉的元素个数
54     cout << "\ngquiz2.erase(4) : " << num << " removed \n";
55     cout << "\tKEY\tELEMENT\n";
56     for (itr = gquiz2.begin(); itr != gquiz2.end(); ++itr)
57     {
58         cout << '\t' << itr->first
59         | | | << '\t' << itr->second << '\n';
60     }
61     cout << endl;
62
63     // lower bound and upper bound
64     // for map gquiz1 key = 5
65     cout << "gquiz1.lower_bound(5) : \tKEY = ";
66     cout << gquiz1.lower_bound(5)->first << '\t';
67     cout << "\tELEMENT = "
68     | | | << gquiz1.lower_bound(5)->second << endl;
69     cout << "gquiz1.upper_bound(5) : \tKEY = ";
70     cout << gquiz1.upper_bound(5)->first << '\t';
71     cout << "\tELEMENT = "
72     | | | << gquiz1.upper_bound(5)->second << endl;
73     cout << endl;
74     return 0;
75 }

```

```
yahui@Yahui:/media/sf_VM$ g++ test.cpp  
yahui@Yahui:/media/sf_VM$ ./a.out
```

The map gquiz1 is :

KEY	ELEMENT
1	40
2	30
3	60
4	20
5	50
6	50
7	10

The map gquiz2 after assign from gquiz1 is :

KEY	ELEMENT
1	40
2	30
3	60
4	20
5	50
6	50
7	10

gquiz2 after removal of elements less than key=3 :

KEY	ELEMENT
3	60
4	20
5	50
6	50
7	10

gquiz2.erase(4) : 1 removed

KEY	ELEMENT
3	60
5	50
6	50
7	10

```
gquiz1.lower_bound(5) :      KEY = 5      ELEMENT = 50  
gquiz1.upper_bound(5) :     KEY = 6      ELEMENT = 50
```

1.3 容器适配器

- 适配器（**adapter**）是标准库中的一个通用概念。本质上，一个适配器是一种机制，能使某种事物的行为看起来像另外一种事物一样。
 - 一个容器适配器接受一种已有的容器类型，使其行为看起来像一种不同的类型。
- 标准库定义了三个容器适配器
 - **stack**栈：满足序列中被删除、检索和修改的项只能是最近插入序列的项。即按照后进先出的原则。
 - **queue**队列：插入只可以在尾部进行，删除、检索和修改只允许从头部进行。按照先进先出原则。
 - **priority_queue**：优先级队列。最高优先级元素总是第一个出列。

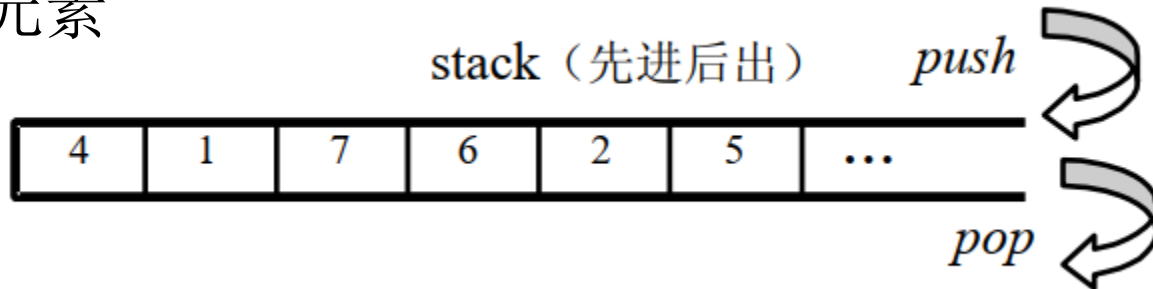
容器适配器: stack

- 可用 vector, list, deque 来实现
 - 缺省情况下, 用 deque 实现

```
template<class T, class Cont = std::deque<T> >  
class stack {  
    ....  
};
```

- push: 插入元素
- pop: 弹出元素
- top: 返回栈顶元素的引用

- stack 是后进先出的数据结构, 只能插入、删除、访问栈顶的元素

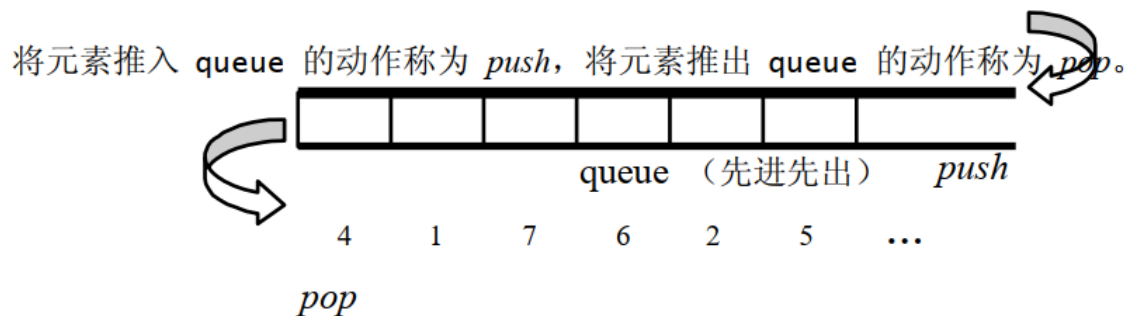


容器适配器: queue

- 和stack基本类似，可以用list和deque实现，缺省情况下用deque实现

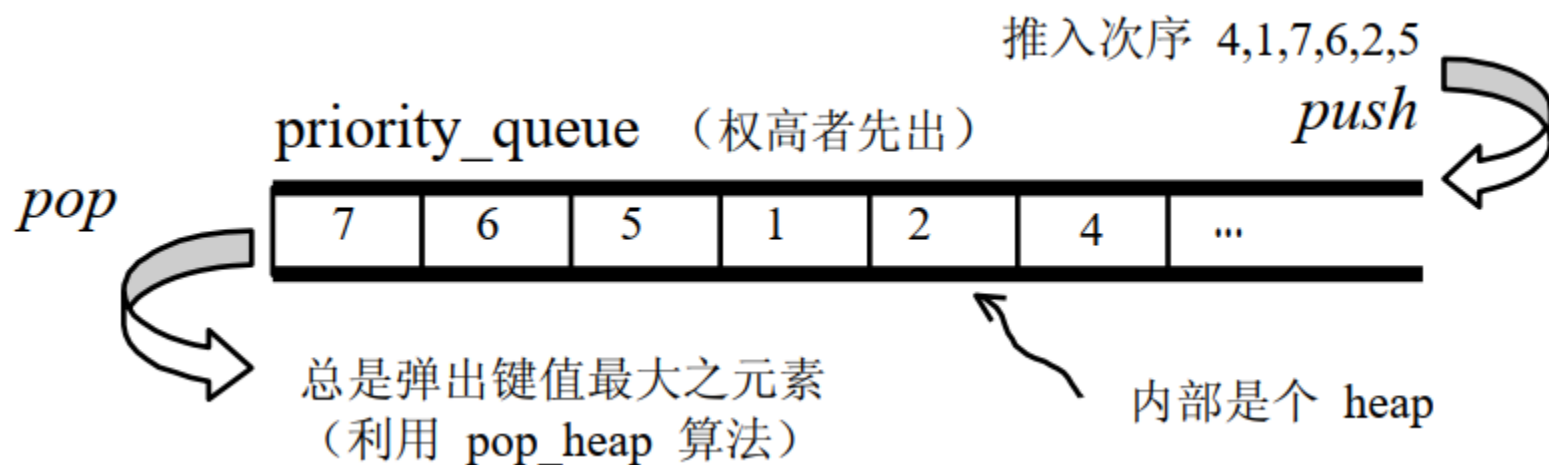
```
template<class T, class Cont = std::deque<T> >  
class queue {  
    ...  
};
```

- 同样也有push,pop,top函数
- push发生在队尾，pop,top发生在队头，先进先出



容器适配器: priority_queue

- 和queue类似，可以用vector和deque实现，缺省情况下用vector实现
- priority_queue 通常用堆排序技术实现，保证最大的元素总是在最前面。即执行pop操作时，删除的是最大的元素，执行top操作时，返回的是最大元素的引用。



```
#include <queue>
#include <iostream>
using namespace std;
int main() {
    priority_queue<double> priorities;
    priorities.push(3.2);
    priorities.push(9.8);
    priorities.push(5.4);
    while( !priorities.empty() ) {
        cout << priorities.top() << " ";
        priorities.pop();
    }
    return 0;
}
```

输出结果:

9.8 5.4 3.2

1.4 使用容器或容器适配器时的一些注意事项

对象被插入容器中时，被插入的是对象的~~一个复制品~~，以vector为例

（注意：vector扩容或缩容的时候，里面的元素会被拷贝一遍）

```
1  #include <iostream>
2  #include <string>
3  #include <vector>
4  using namespace std;
5
6  class A
7  {
8  public:
9      A(const A &a) // 用A对象初始化A对象 (i+1)
10     {
11         i = a.i + 1;
12         cout << "Initialization type 1: " << i << endl;
13     }
14     A(int x) : i(x)
15     {
16         cout << "Initialization type 2: " << i << endl;
17     } // 用int初始化A对象 (i)
18     int i;
19 };
20
21 int main()
22 {
23     vector<A> vec;
24     A a1(10); // a1.i=10
25     A a2(20); // a2.i=20
26     vec.push_back(a1); // vec[0].i=11 因为void push_back( const T& value ); 对象被插入容器中时， 被插入的是对象的一个复制品，此处vec[0]是一个用a1初始化的a对象
27     vec.push_back(a2); // vec[1].i=21
28     cout << "-----" << endl;
29     const A &a3 = vec[0]; // a3.i=12
30     A a4 = vec[1]; // a4.i=22
31     cout << "-----" << endl;
32     cout << a1.i << " " << a2.i << " "
33         << a3.i << " " << a4.i << endl;
34 }
```

```
yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
Initialization type 2: 10
Initialization type 2: 20
Initialization type 1: 11
Initialization type 1: 21
Initialization type 1: 12
-----
Initialization type 1: 22
-----
10 20 12 22
```

容器中的对象含有指针元素时，要注意double free问题：

```
1  #include <queue>
2  using namespace std;
3
4  class Test{
5      int *myArray;
6
7      public:
8      Test(){
9          myArray = new int[10];
10     }
11
12     ~Test(){
13         delete[] myArray;
14     }
15
16 };
17
18
19 int main(){
20     queue<Test> q;
21     Test t;
22     q.push(t);
23 }
```

```
yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
free(): double free detected in tcache 2
Aborted (core dumped)
```

（Ubuntu系统下，Window系统下不报错）

解决方案：

```
1  #include <queue>
2  #include <vector>
3
4  class Test
5  {
6      std::vector<int> myArray;
7
8      public:
9      Test() : myArray(10)
10     {
11     }
12 };
13
14 int main()
15 {
16     std::queue<Test> q;
17     Test t;
18     q.push(t);
19 }
```

本章内容

1. 标准模板库的容器
2. 标准模板库的迭代器
3. 标准模板库的算法

2. 迭代器

- 提供一种方法访问一个容器中的各个元素，而又不需暴露该容器的内部细节
- 迭代器用法和指针类似
 - 定义一个容器类的迭代器： 容器类名::iterator 变量名；
 - 访问一个迭代器指向的元素：*迭代器变量名
 - 迭代器上可以执行++操作，指向容器中的下一个元素
 - 若迭代器到达了容器中的最后一个元素的后面，则迭代器变成past-the-end值，相当于NULL指针
- 有const和非const两种
 - 通过const迭代器可以读取它指向的元素
 - 通过非const迭代可以读取、修改其指向的元素

为何使用迭代器

1、在double数组中搜索特定值

```
double * find_ar(double * at, int n, const double & val) {  
    for (int i = 0; i < n; i++)  
        if (at[i] == val)  
            return &ar[i];  
    return nullptr;  
}
```

2、在自定义链表中搜索特定值

```
Node * find_ll(Node * head, const double & val) {  
    Node * start;  
    for (start = head; start != nullptr; start = start->next )  
        if (start->item == val)  
            return start;  
    return nullptr;  
}
```

STL中迭代器的分类

- STL中的迭代器按功能由弱到强分为5种：
 1. 输入：Input iterators 提供对数据的只读访问
 1. 输出：Output iterators 提供对数据的只写访问
 2. 正向：Forward iterators 提供读写操作，并能一次一个地向前推进迭代器
 3. 双向：Bidirectional iterators提供读写操作，并能一次一个地向前和向后移动
 4. 随机访问：Random access iterators提供读写操作，并能在数据中随机移动。
- 编号大的迭代器拥有编号小的迭代器的所有功能，能当作编号小的迭代器使用

不同类别迭代器所能进行的操作

- 所有迭代器: `++p, p++`
- 输入迭代器: `(x = ) *p, p = p1, p == p1, p != p1`
- 输出迭代器: `*p ( = x), p = p1`
- 正向迭代器: 上面全部
- 双向迭代器: 上面全部, `--p, p--`,
- 随机访问迭代器: 上面全部, 以及:
 - `p += i, p -= i`,
 - `p + i`: 返回指向 `p` 后面的第 `i` 个元素的迭代器
 - `p - i`: 返回指向 `p` 前面的第 `i` 个元素的迭代器
 - `p[i]`: `p` 后面的第 `i` 个元素的引用
 - `p < p1, p <= p1, p > p1, p >= p1`

容器所支持的迭代器类别

- 不同容器上支持的迭代器功能强弱有所不同。
- 容器的迭代器的功能强弱，决定了该容器是否支持STL中的某种算法。
 - 例：排序算法需要通过随机迭代器来访问容器中的元素，那么有的容器就不支持排序算法；搜索算法无需随机访问元素，只要支持迭代器的容器就能够支持搜索算法

容器	迭代器类别
vector	随机
deque	随机
list	双向
set/multiset	双向
map/multimap	双向
stack	不支持迭代器
queue	不支持迭代器
priority_queue	不支持迭代器

容器所支持的迭代器类别

vector的迭代器是随机迭代器，所以遍历 vector 可有以下几种做法：

```
vector<int> v(100);
```

```
int i;
```

```
for(i = 0; i < v.size() ; i ++)
```

```
    cout << v[i];
```

```
vector<int>::const_iterator ii;
```

```
for( ii = v.begin(); ii != v.end (); ii ++ )
```

```
    cout << * ii;
```

```
ii = v.begin();
```

```
while( ii < v.end()) { //间隔一个输出
```

```
    cout << * ii;
```

```
    ii = ii + 2;
```

```
}
```

容器所支持的迭代器类别

而 list 的迭代器是双向迭代器，所以以下代码可以：

```
list<int> v;  
list<int>::const_iterator ii;  
for( ii = v.begin(); ii != v.end(); ii ++ )  
    cout << * ii;
```

以下代码则不行：

```
for( ii = v.begin(); ii < v.end(); ii ++ )  
    cout << * ii;  
//双向迭代器不支持 <
```

```
for(int i = 0; i < v.size() ; i ++)  
    cout << v[i]; //双向迭代器不支持 []
```

本章内容

1. 标准模板库的容器
2. 标准模板库的迭代器
3. 标准模板库的算法

3. 算法

- STL中提供能在各种容器中通用的算法，比如插入，删除，查找，排序等。
 - 算法就是一个个函数模板
 - 算法通过迭代器来操纵容器中的元素。许多算法需要两个参数，一个是起始元素的迭代器，一个是终止元素的后面一个元素的迭代器。比如，排序和查找。
 - 有的算法返回一个迭代器。比如 `find()` 算法，在容器中查找一个元素，并返回一个指向该元素的迭代器。
 - 算法可以处理容器，也可以处理C语言的数组。

算法分类

- 变化序列算法
 - copy, remove, fill, replace, random_shuffle, swap,
 - 会改变容器
- 非变化序列算法：
 - adjacent_find, equal, mismatch, find, count, search, count_if, for_each, search_n

以上函数模板都在`<algorithm>` 中定义

此外还有其他算法，比如`<numeric>`中的算法

排序和查找算法

- Sort

```
template<class RandomIt>  
void sort(RandomIt first, RandomIt last);
```

- find

```
template<class InIt, class T>  
InIt find(InIt first, InIt last, const T& val);
```

- binary_search 二分查找，要求容器已经有序

```
template<class FwdIt, class T>  
bool binary_search(FwdIt first, FwdIt last, const T& val);
```

算法示例： find()

```
template<class InIt, class T>
```

```
InIt find(InIt first, InIt last, const T& val);
```

- **first** 和 **last** 这两个参数都是容器的迭代器，它们给出了容器中的查找区间起点和终点。
 - 这个区间是个左闭右开的区间，即区间的起点是位于查找范围之中的，而终点不是
- **val**参数是要查找的元素的值
- 函数返回值是一个迭代器。如果找到，则该迭代器指向被找到的元素。如果找不到，则该迭代器指向查找区间终点。

```

#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;
int main() {
    int array[10] = {10,20,30,40};
    vector<int> v;
    v.push_back(1); v.push_back(2);
    v.push_back(3); v.push_back(4);
    vector<int>::iterator p;
    p = find(v.begin(), v.end(), 3);
    if( p != v.end())
        cout << * p << endl;
    p = find(v.begin(), v.end(), 9);
    if( p == v.end())
        cout << "not found " << endl;
    p = find(v.begin()+1, v.end()-2, 1); // 没找到
    if( p != v.end())
        cout << * p << endl;
    int * pp = find(array, array+4, 20);
    cout << * pp << endl;
}

```

输出：

3

not found

3

20

算法示例： sort()

```
int main() {  
    const int SIZE = 10;  
    int a1[] = { 2,8,1,50,3,100,8,9,10,2 };  
    vector<int> v(a1,a1+SIZE);  
    vector<int>::iterator location;  
    location = find(v.begin(),v.end(),10);  
    if( location != v.end()) {  
        cout << "1) " << location - v.begin() << endl;  
    }  
    sort(v.begin(), v.end());  
    if( binary_search(v.begin(),v.end(),9))  
        cout << "2) " << "9 found" << endl;  
    else  
        cout << "3) " << "9 not found" << endl;  
    return 0;  
}
```

输出：（无sort语句）

1) 8
3) 9 not found

输出：（有sort语句）

1) 8
2) 9 found

算法示例： rand()

```
int x = 100;
```

```
int num = rand() % x; // 生成一个[0,x-1]区间的随机整数
```

- rand()的内部实现是用线性同余法（一种产生伪随机数的方法）做的，它不是真的随机数，只不过是因为其周期特别长，所以在一定的范围里可看成是随机的。
- rand()产生的是伪随机数，每次执行时是相同的。若要不同，以不同的值来初始化它，初始化的函数就是 srand()。
- srand((unsigned)time(NULL)); // 使用当前时间使随机数发生器随机化
- rand() 会返回一随机数值，范围在 0 至 RAND_MAX 间。

Windows平台：#define RAND_MAX 0x7FFF，即：32767

Linux平台：#define RAND_MAX 0x7FFFFFFF，即：2147483647

算法示例： rand()

```
1  #include <iostream>
2  #include <ctime>
3  using namespace std;
4  int main()
5  {
6      srand((unsigned)time(NULL));
7
8      int max_num = 1e7, rand_times = 1e3;
9      int small_num_times = 0, large_num_times = 0;
10
11     for (int counter = 0; counter < rand_times; counter++)
12     {
13         int rand_num = rand() % max_num;
14         if (rand_num < max_num / 2)
15         {
16             small_num_times++;
17         }
18         else
19         {
20             large_num_times++;
21         }
22     }
23     cout << small_num_times << "    " << large_num_times << endl;
24     return 0;
25 }
```

```
PS C:\Users\Yahui\Drive\VM>
1000    0
```

```
yahui@Yahui:/media/sf_VM$ g++ test.cpp
yahui@Yahui:/media/sf_VM$ ./a.out
486    514
```

<https://www.boost.org/>

Boost库提供了更多的随机数生成算法。

Boost库是一个可移植、提供源代码的C++库，作为标准库的后备，是C++标准化进程的开发引擎之一。

Boost库由C++标准委员会库工作组成员发起，在C++社区中影响甚大，是“准”标准库。

Boost库——斐波那契堆



WELCOME TO BOOST.ORG!

Boost provides free peer-reviewed portable C++ source libraries.

We emphasize libraries that work well with the C++ Standard Library. Boost libraries are intended to be widely useful, and usable across a broad spectrum of applications. The [Boost license](#) encourages the use of Boost libraries for all users with minimal restrictions.

We aim to establish "existing practice" and provide reference implementations so that Boost libraries are suitable for eventual standardization. Beginning with the ten Boost Libraries included in the Library Technical Report (TR1) and continuing with every release of the ISO standard for C++ since 2011, the [C++ Standards Committee](#) has continued to rely on Boost as a valuable source for additions to the Standard C++ Library.

BOOST AS AN ORGANIZATION

The Boost organization and wider Boost community supports research and education into the best possible uses of C++ and libraries developed for it, particularly, but not exclusively those contained in the Boost Library.

The organization and community support mailing lists and chat rooms to educate about best practices and cutting edge techniques for the user of Boost Libraries and C++ in general.

Since 2006 an intimate week long annual conference related to Boost called [C++ Now](#) has been held in Aspen, Colorado each May. The conference is an educational opportunity focused on cutting-edge C++. Boost has been a participant in the annual Google Summer of Code since 2007, in which students develop their skills by working on Boost Library development.

DOWNLOADS

CURRENT RELEASE

- [Version 1.82.0](#)
[Release Notes](#) | [Download](#) | [Documentation](#)
April 14th, 2023 03:08 GMT

[More Downloads...](#) (RSS)

NEWS

◦ [Version 1.82.0](#)

New Libraries: MySQL. Updated Libraries: Any, Asio, Atomic, Beast, ContainerHash, Core, DLL, Filesystem, Geometry, Histogram, JSON, Locale, Math, MultiIndex, Multiprecision, Nowide, PFR, Process, Stacktrace, StaticString, Unordered, URL, Variant.

April 14th, 2023 03:08 GMT

◦ [Version 1.81.0](#)

New Libraries: URL. Updated Libraries: Beast, ContainerHash, Core, Describe, DLL, Filesystem, Fusion, Geometry, Histogram, Iterator, JSON, LexicalCast, Locale, Nowide, Stacktrace, PFR, STLInterfaces, System, Unordered, Variant, Variant2.

December 14th, 2022 17:44 GMT

◦ [Version 1.80.0](#)

Updated Libraries: Asio, Atomic, Filesystem, GIL, Histogram, Iterator, JSON, LEAF, Locale, Log, Math, Multiprecision, Multi-index Containers, Nowide, Optional, STLInterfaces, System, Unordered, Utility, Wave.

August 10th, 2022 21:25 GMT

```
g++ -I/home/boost_1_75_0 test.cpp
```

下载的Boost库文件夹的地址

```

1  #include <iostream>
2  #include <boost/heap/fibonacci_heap.hpp> // defferent types of heap
3  using namespace std;
4
5  struct node {
6      int vertex;
7      float priority_value;
8  }; // define the node in the queue
9  bool operator<(node const& x, node const& y) {
10     return x.priority_value > y.priority_value; // < is the max-heap; > is the min heap
11 }
12 typedef typename boost::heap::fibonacci_heap<node>::handle_type handle_t; // define the handle
13
14 int main() {
15     handle_t handle[5]; // the handle of each node in the queue
16     boost::heap::fibonacci_heap<node> pq; // define a queue
17
18     node a;
19     a.priority_value = 1.1; a.vertex = 1;
20     handle[1] = pq.push(a); //enqueue a node
21     a.priority_value = 2.1; a.vertex = 2;
22     handle[2] = pq.push(a);
23     //pq.erase(handle[1]); // remove a node
24     pq.pop(); // dequeue the top element
25
26     boost::heap::fibonacci_heap<node> pq2;
27     a.priority_value = 3.1; a.vertex = 3;
28     handle[3] = pq2.push(a);
29     a.priority_value = 4.1; a.vertex = 4;
30     handle[4] = pq2.push(a);
31     pq.merge(pq2); // merge two queues
32
33     a.priority_value = 0.2; a.vertex = 4;
34     pq.update(handle[4], a);
35
36     cout << "Priority Queue iteration: " << endl;
37     for (boost::heap::fibonacci_heap<node>::iterator it = pq.begin(); it != pq.end(); ++it) {
38         a = *it;
39         cout << a.vertex << " "; // 1, 2, 3 in unspecified order
40         cout << a.priority_value << " " << endl; // 1, 2, 3 in unspecified order
41     }
42     cout << endl;
43     cout << "pq size=" << pq.size() << endl;
44     cout << "pq top vertex=" << pq.top().vertex << endl;
45     cout << "pq top priority_value=" << pq.top().priority_value << endl;
46 }
47

```

Microsoft Visual Studio Debug Co

Priority Queue iteration:

2 2.1

4 0.2

3 3.1

pq size=3

pq top vertex=4

pq top priority_value=0.2

作业

- YOJ-626. 模板与迭代器
- YOJ-627. 模板与链式迭代器
- YOJ-628. STL容器练习（1）
- YOJ-629. STL容器练习（2）
- YOJ-630. STL容器集合处理